EE4077 FUNDAMENTALS OF MACHINE LEARNING

Week 7:
Nearest Neighbors, Decision Trees

OUTLINE

- Nearest neighbor (NN) classifier
- Practical aspects of NN
- Decision trees

Parametric vs. Nonparametric

- So far, we've discussed parametric machine learning models:
  - Linear regression
  - Naive Bayes
  - Logistic regression
  - Linear SVMs
- Now we will discuss two *nonparametric* models:
  - Nearest neighbors
  - Decision trees

Parametric vs. Nonparametric

Key difference:

- Parametric models assume that the data can be characterized via some fixed set of parameters θ. Given this set of parameters, our future predictions are independent of the data $\mathcal{D}$, i.e., $P(x|\theta, \mathcal{D}) = P(x|\theta)$.
  - Often simpler and faster to learn, but can sometimes be a poor fit
- Nonparametric models instead assume that the model features depend on the data $\mathcal{D}$. The number of features tends to grow with the size of the dataset.
  - More complex and expensive, but can learn more flexible patterns
- Both parametric and non-parametric methods can be used for either regression or classification.

Recognizing flowers

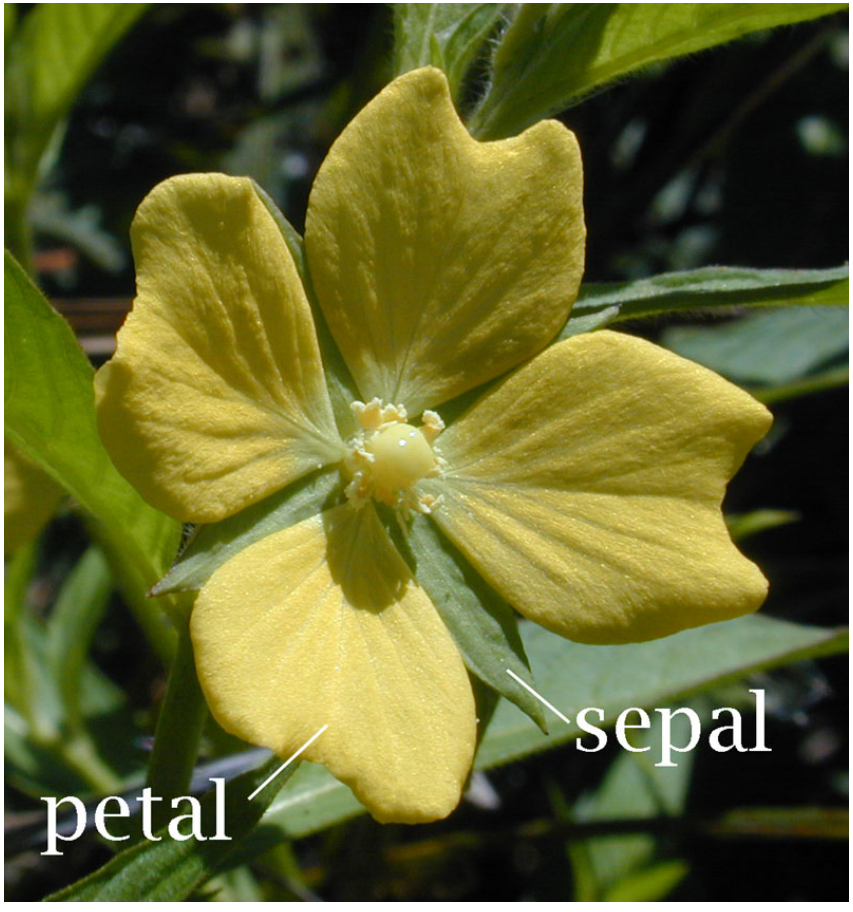
Types of Iris: setosa, versicolor, and virginica





Measuring the properties of the flowers

Features: the widths and lengths of sepal and petal



Often, data is conveniently organized as a table

Ex: Iris data (click here for all data)

- 4 features
- 3 classes

Fisher's *Iris* Data

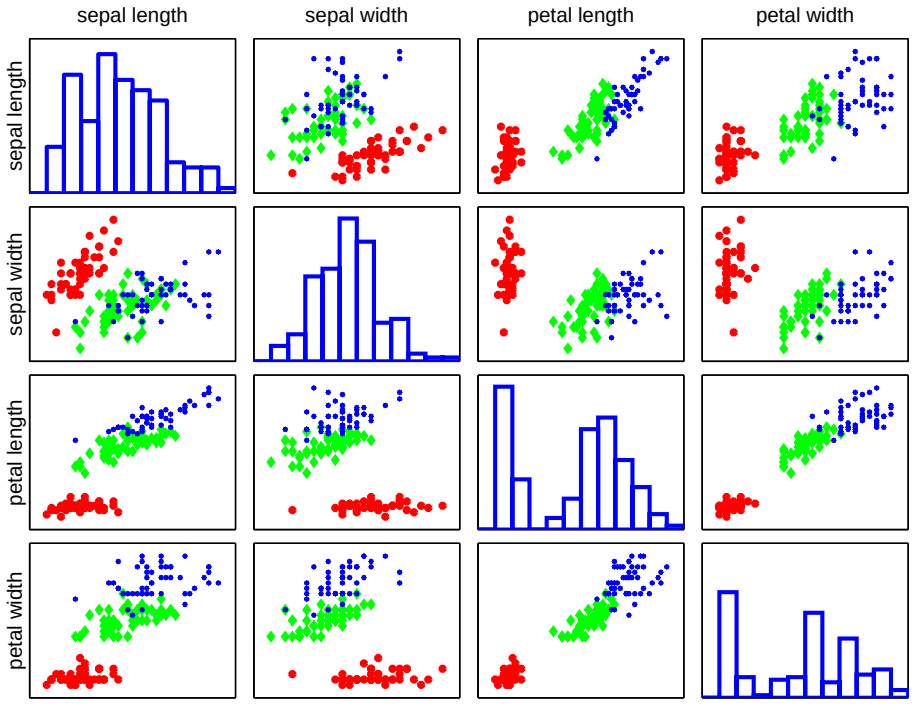
Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	*I. setosa*
4.9	3.0	1.4	0.2	*I. setosa*
4.7	3.2	1.3	0.2	*I. setosa*
4.6	3.1	1.5	0.2	*I. setosa*
5.0	3.6	1.4	0.2	*I. setosa*
5.4	3.9	1.7	0.4	*I. setosa*
4.6	3.4	1.4	0.3	*I. setosa*
5.0	3.4	1.5	0.2	*I. setosa*
4.4	2.9	1.4	0.2	*I. setosa*
4.9	3.1	1.5	0.1	*I. setosa*

Pairwise scatter plots of 131 flower specimens

Visualization of data helps to identify the right learning model

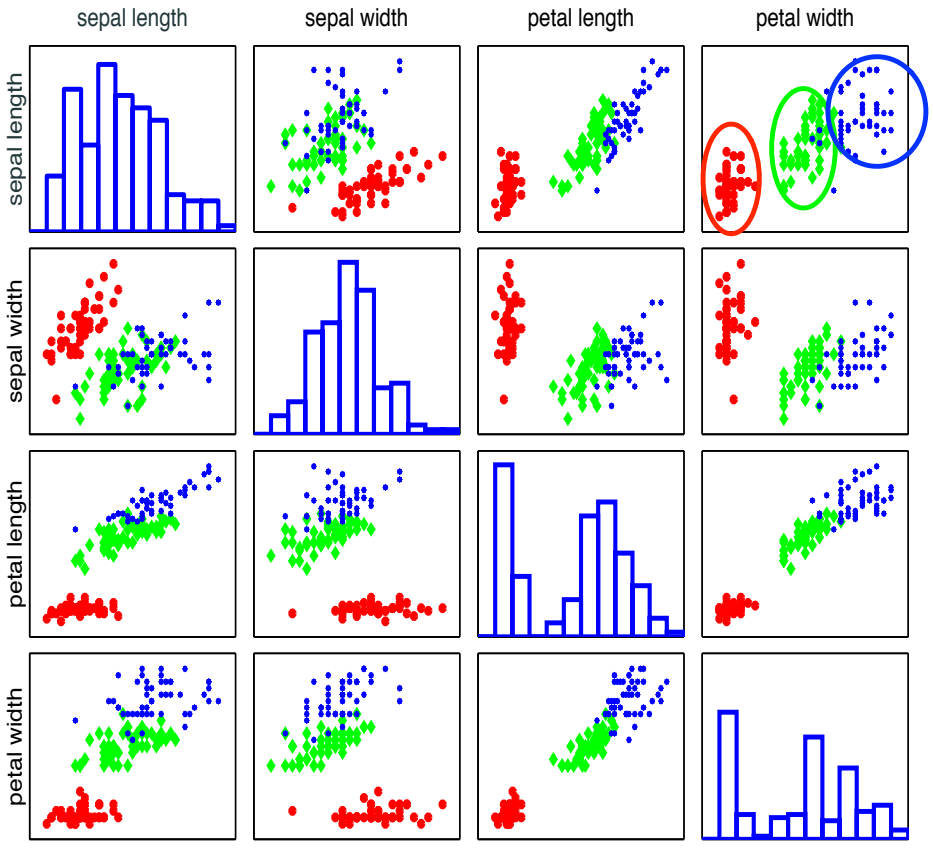
Which combination of features separates the three classes?

Figure 1: Each colored point is a flower specimen: setosa, versicolor, virginica

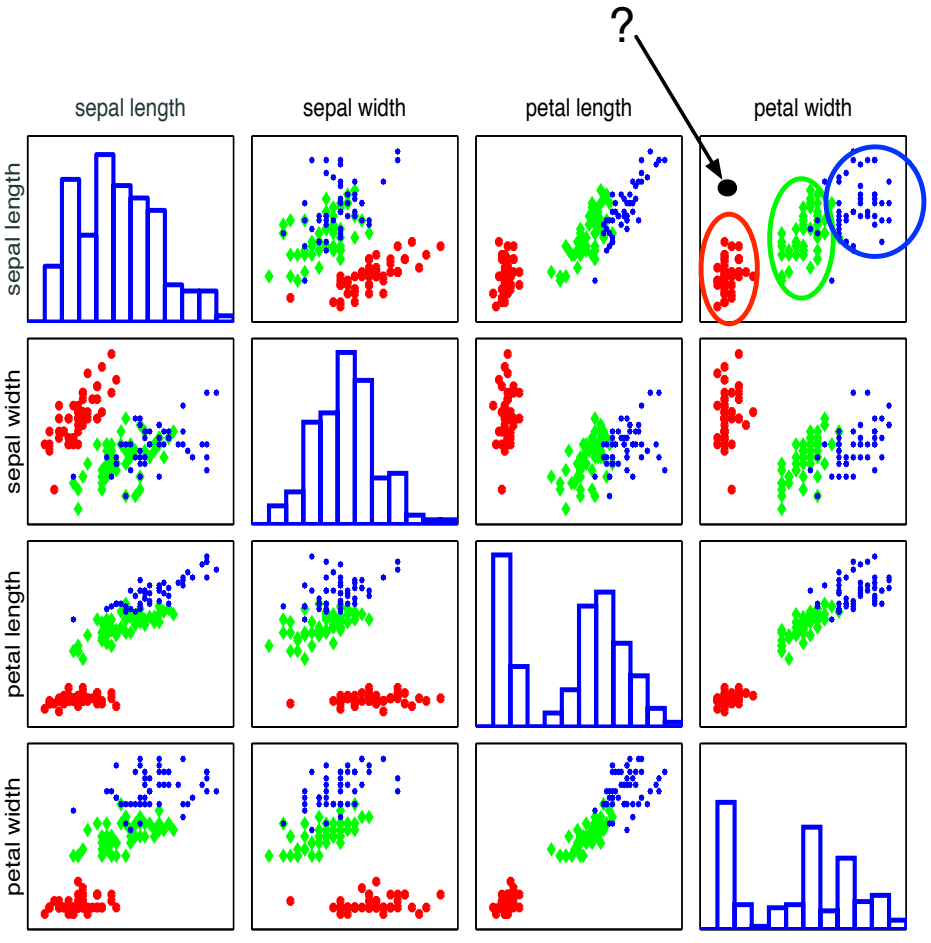


Different types seem well-clustered and separable

Using two features: petal width and sepal length



Labeling an unknown flower type



Closer to red cluster: so labeling it as **setosa**

Multi-class classification

Classify data into one of the multiple categories

- Input (feature vectors): $\boldsymbol{x} \in \mathbb{R}^{\mathsf{D}}$
- Output (label): $y \in [\mathsf{C}] = \{1, 2, \cdots, \mathsf{C}\}$
- Learning goal: $y = f(\boldsymbol{x})$

Recall special case: binary classification

- Number of classes: $\mathsf{C} = 2$
- Labels: $\{0, 1\}$ or $\{-1, +1\}$

More terminology

Training data (set)

- N samples/instances: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_N, y_N)\}$
- They are used for learning $f(\cdot)$

Test (evaluation) data

- M samples/instances: $\mathcal{D}^{\text{TEST}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_M, y_M)\}$
- They are used for assessing how well $f(\cdot)$ will do in predicting an unseen $\boldsymbol{x} \notin \mathcal{D}^{\text{TRAIN}}$

Training data and test data should *not* overlap: $\mathcal{D}^{\text{TRAIN}} \cap \mathcal{D}^{\text{TEST}} = \emptyset$

Nearest neighbor classification (NNC)

Nearest neighbor of a (training or test) data point

$$\boldsymbol{x}(1) = \boldsymbol{x}_{\mathrm{nn}(\boldsymbol{x})}$$

where $\mathrm{nn}(\boldsymbol{x}) \in [\mathrm{N}] = \{1, 2, \cdots, \mathrm{N}\}$, i.e., the index to one of the training instances

$$\mathrm{nn}(\boldsymbol{x}) = \mathrm{argmin}_{n \in [\mathrm{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2 = \mathrm{argmin}_{n \in [\mathrm{N}]} \sum_{d=1}^{\mathrm{D}} (x_d - x_{nd})^2$$

Classification rule

$$y = f(\boldsymbol{x}) = y_{\mathrm{nn}(\boldsymbol{x})}$$

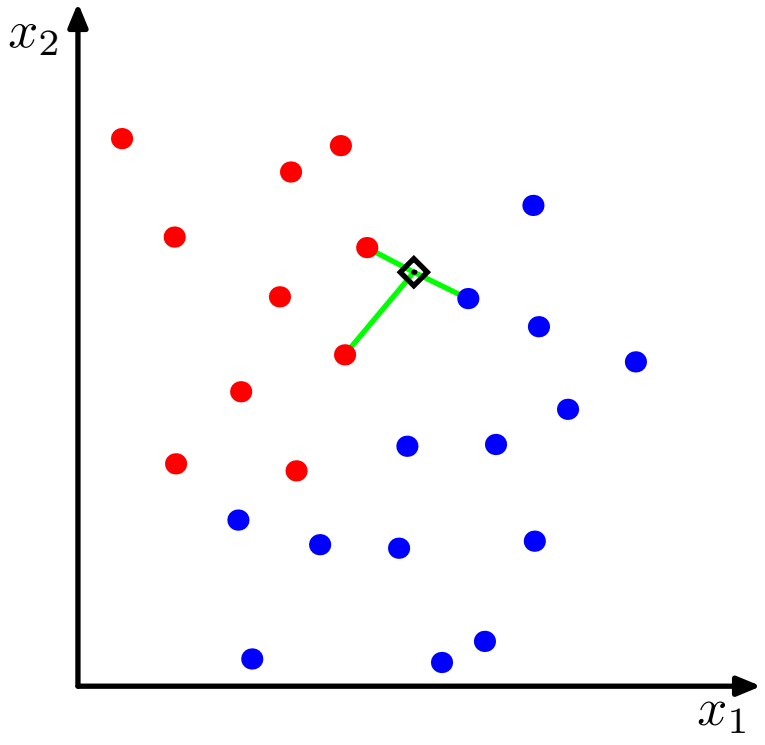
Example: if $\mathrm{nn}(\boldsymbol{x}) = 2$, then

$$y_{\mathrm{nn}(\boldsymbol{x})} = y_2,$$

which is the label of the 2nd data point.

Visual example

In this 2-dimensional example, the nearest point to $\boldsymbol{x}$ is a red training instance, thus, $\boldsymbol{x}$ will be labeled as red.



(a)

Example: classify Iris with two features

Training data

ID (n)	petal width (x_1)	sepal length (x_2)	category (y)
1	0.2	5.1	setosa
2	1.4	7.0	versicolor
3	2.5	6.7	virginica

Flower with unknown category

petal width $= 1.8$ and sepal length $= 6.4$

Calculating distance from (x_1, x_2) to (x_{n1}, x_{n2}): $(x_1 - x_{n1})^2 + (x_2 - x_{n2})^2$

ID	distance
1	4.25
2	0.52
3	0.58

Thus, the predicted category is 2 (*versicolor*)

How to measure "nearness" with other distances?

Previously, we used the Euclidean distance

$$\text{nn}(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$$

We can also use alternative distances

E.g., the following L_1 distance (i.e., city block distance, or Manhattan distance)

$$\begin{aligned}\text{nn}(\boldsymbol{x}) &= \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_1 \\ &= \text{argmin}_{n \in [\text{N}]} \sum_{d=1}^{\text{D}} |x_d - x_{nd}|\end{aligned}$$

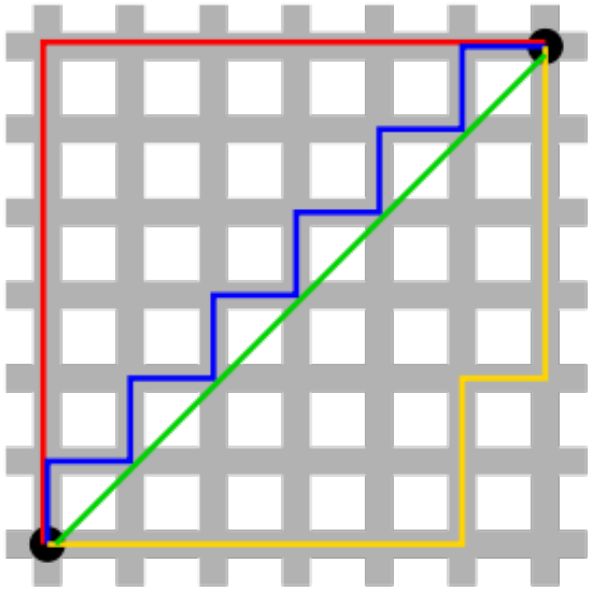
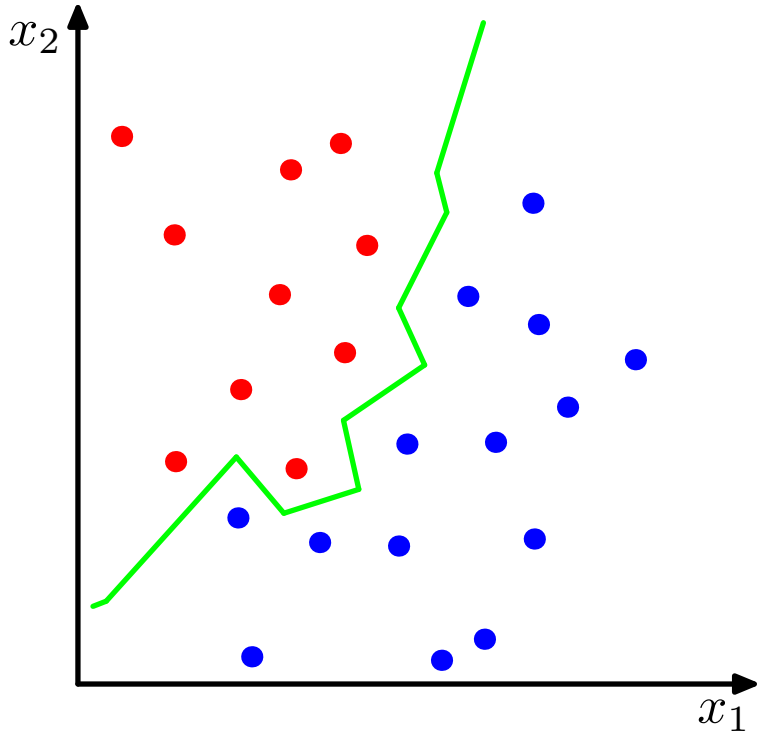


Figure 2: Green line is Euclidean distance. Red, Blue, and Yellow lines are L_1 distance

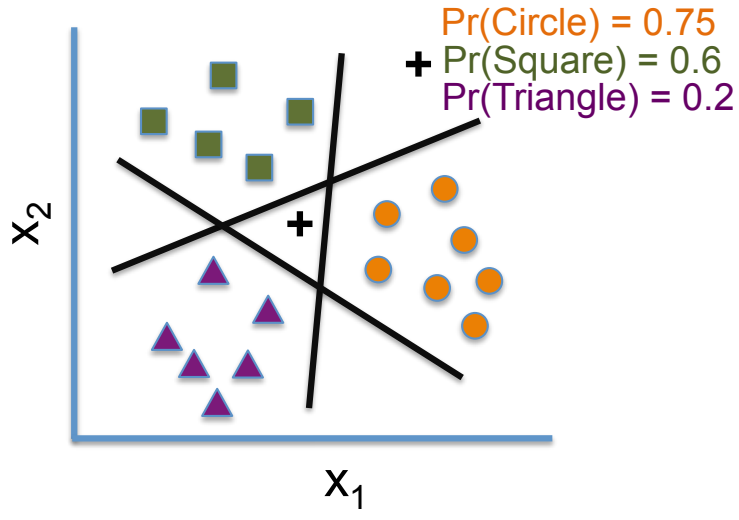
Decision boundary

For every point in the space, we can determine its label using the NNC rule. This gives rise to a *decision boundary* that partitions the space into different regions.



(b)

Recall: Multi-class Classification



Previously, we learned a multi-class classifier by combining binary, linear decision boundaries to partition the feature space.

Parametric vs. Nonparametric, Revisited

Nonparametric models instead assume that the model features depend on the data $\mathcal{D}$. The number of features tends to grow with the size of the dataset.

- Parametric models are often simpler and faster to learn, but can sometimes be a poor fit
- Nonparametric models are more complex and expensive, but can learn more flexible patterns

How does this manifest for nearest neighbors?

- Nearest neighbors often learns a *highly nonlinear* decision boundary.
- But, we need to compare the test data point to *every sample in the training dataset*.

K-nearest neighbor (KNN) classification

Increase the number of nearest neighbors to use?

- 1-nearest neighbor: $\text{nn}_1(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}]} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$
- 2nd-nearest neighbor: $\text{nn}_2(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}] - \text{nn}_1(\boldsymbol{x})} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$
- 3rd-nearest neighbor: $\text{nn}_2(\boldsymbol{x}) = \text{argmin}_{n \in [\text{N}] - \text{nn}_1(\boldsymbol{x}) - \text{nn}_2(\boldsymbol{x})} \|\boldsymbol{x} - \boldsymbol{x}_n\|_2^2$

The set of K-nearest neighbors

$$\text{knn}(\boldsymbol{x}) = \{\text{nn}_1(\boldsymbol{x}), \text{nn}_2(\boldsymbol{x}), \cdots, \text{nn}_K(\boldsymbol{x})\}$$

Let $\boldsymbol{x}(k) = \boldsymbol{x}_{\text{nn}_k(\boldsymbol{x})}$, then

$$\|\boldsymbol{x} - \boldsymbol{x}(1)\|_2^2 \leq \|\boldsymbol{x} - \boldsymbol{x}(2)\|_2^2 \cdots \leq \|\boldsymbol{x} - \boldsymbol{x}(K)\|_2^2$$

How to classify with K neighbors?

Classification rule

- Every neighbor votes: suppose y_n (the true label) for $\boldsymbol{x}_n$ is c, then
  - vote for c is 1
  - vote for $c' \neq c$ is 0

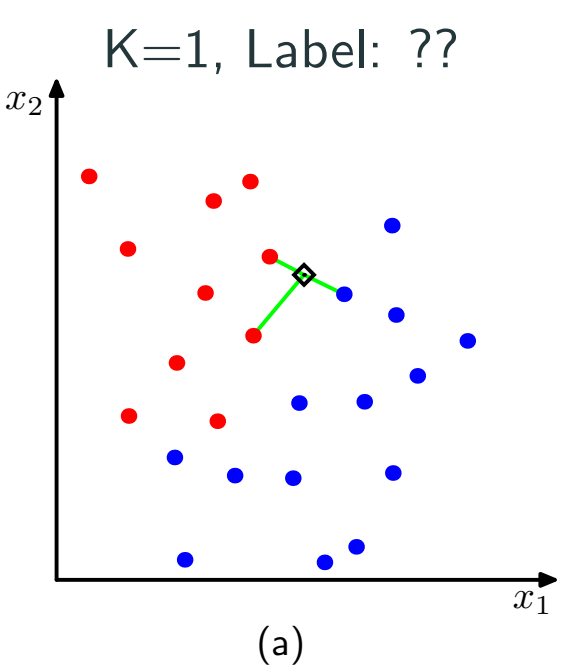
  We use the *indicator function* $\mathbb{I}(y_n == c)$ to represent the votes.
- Aggregate everyone's vote

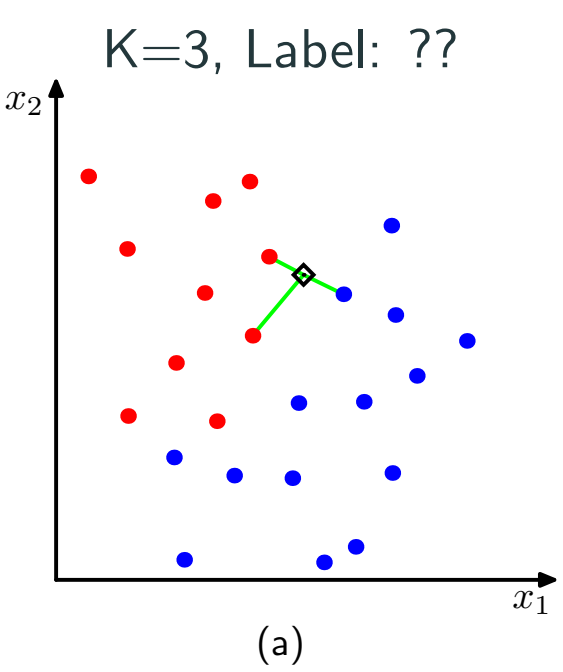
$$v_c = \sum_{n \in \text{knn}(\boldsymbol{x})} \mathbb{I}(y_n == c), \quad \forall \quad c \in [\text{C}]$$

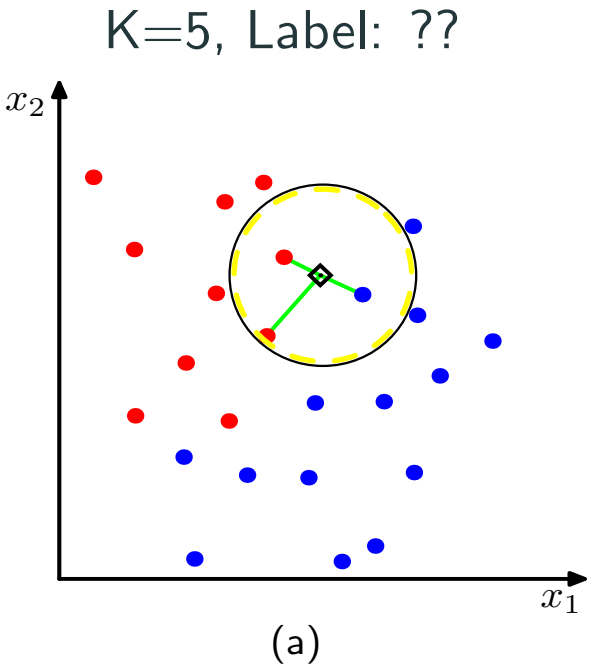
- Label with the majority, breaking ties arbitrarily

$$y = f(\boldsymbol{x}) = \arg\max_{c \in [\text{C}]} v_c$$

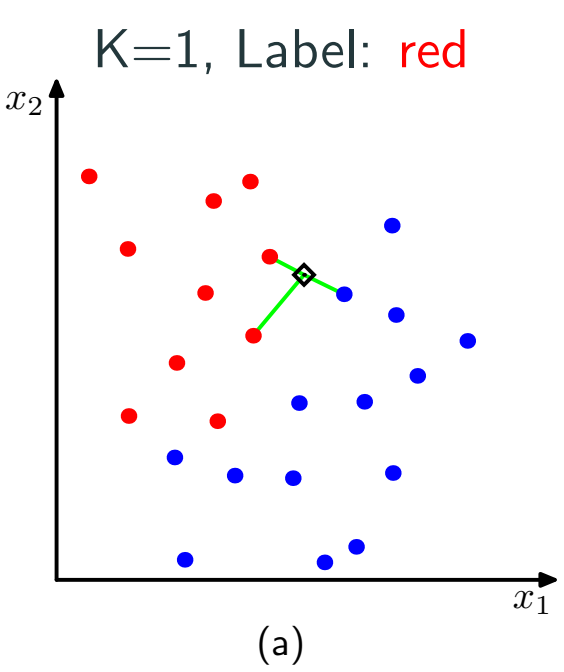
Example



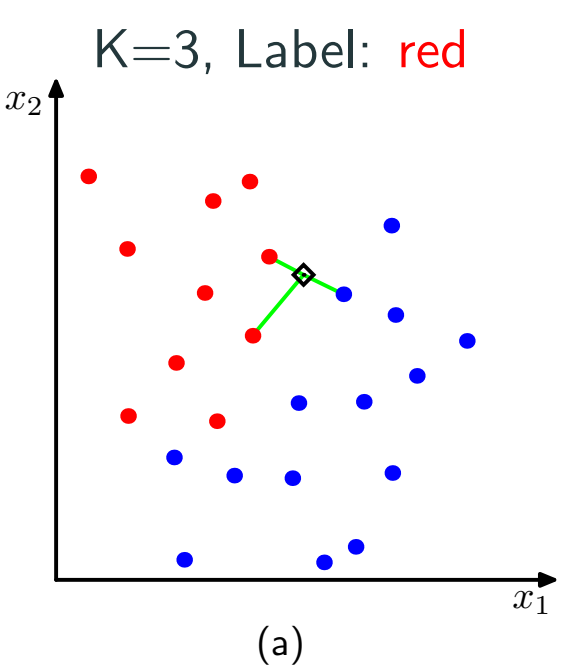




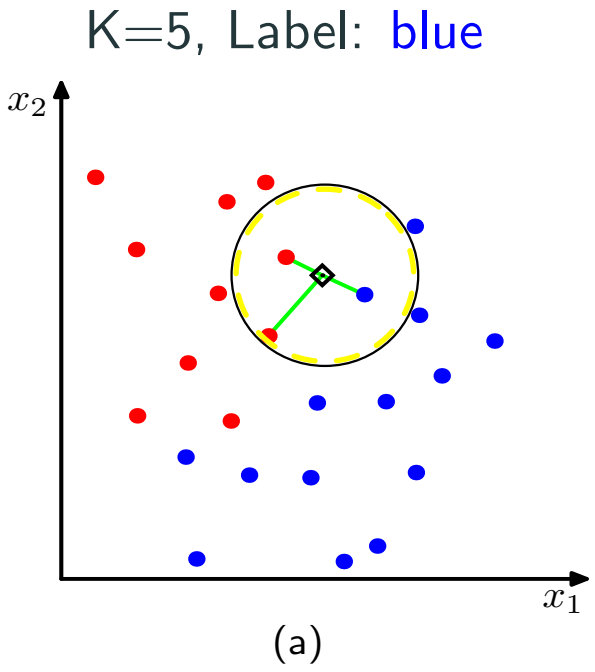
Example



(a)

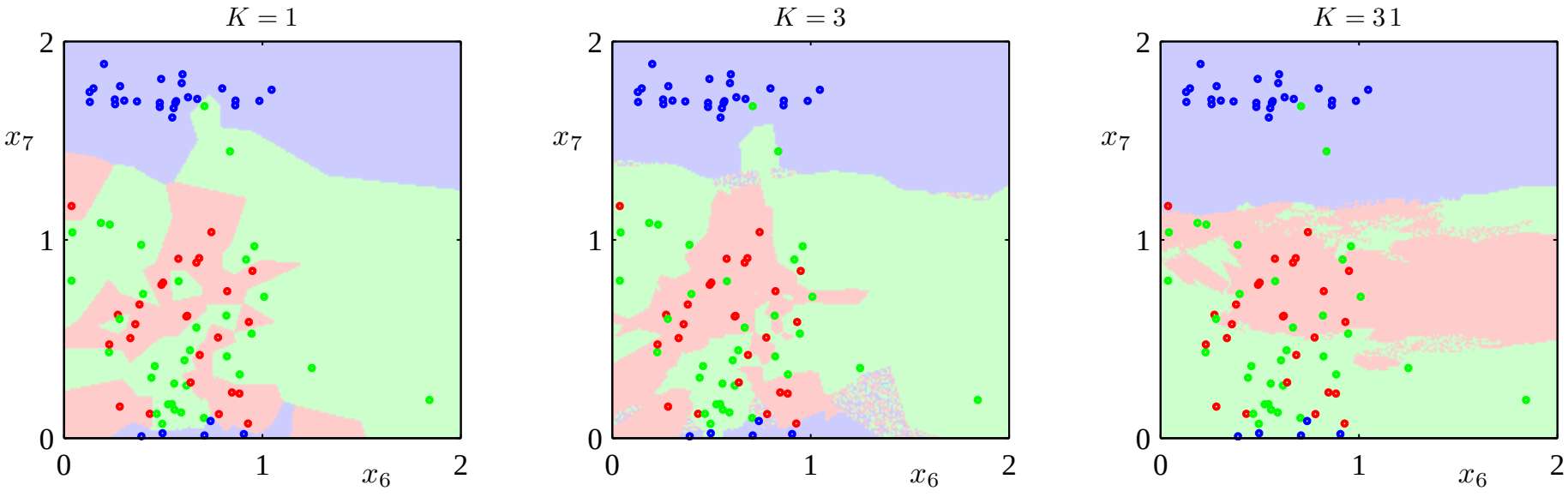


(a)



(a)

How to choose an optimal K?



When K increases, the decision boundary becomes smooth.

Why use nearest neighbors?

Advantages of NNC

- Computationally, simple and easy to implement – just compute distances, no optimization required
- Can learn complex decision boundaries

Disadvantages of NNC

- Computationally intensive for large-scale problems: $O(\mathrm{N}D)$ for labeling a data point
- We need to "carry" the training data around. Without it, we cannot do classification. This type of method is called *nonparametric*.
- Choosing the right distance measure and K can be difficult.

Hyperparameters in NN

Two crucial choices for NN

- Choosing K, i.e., the number of nearest neighbors (default is 1)
- Choosing the right distance measure (default is Euclidean distance), for example, from the following generalized distance measure

$$\|\boldsymbol{x} - \boldsymbol{x}_n\|_p = \left(\sum_d |x_d - x_{nd}|^p \right)^{1/p}$$

for $p \geq 1$.

These are not specified by the algorithm itself — resolving them requires empirical studies and are task/dataset-specific.

Hyperparameter tuning on a validation dataset

Training data

- N samples/instances: $\mathcal{D}^{\text{TRAIN}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_N, y_N)\}$
- They are used for learning $f(\cdot)$

Test data

- M samples/instances: $\mathcal{D}^{\text{TEST}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_M, y_M)\}$
- They are used for assessing how well $f(\cdot)$ will do in predicting an unseen $\boldsymbol{x} \notin \mathcal{D}^{\text{TRAIN}}$

Validation data

- L samples/instances: $\mathcal{D}^{\text{VAL}} = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \cdots, (\boldsymbol{x}_L, y_L)\}$
- They are used to optimize hyperparameter(s).

Training data, validation and test data should *not* overlap!

Recipe

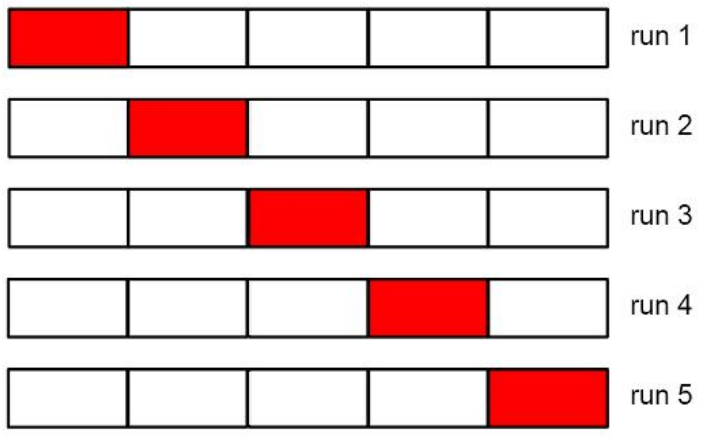
- For each possible value of the hyperparameter (say $K = 1, 3, \cdots, 100$)
  - Train a model using $\mathcal{D}^{\text{TRAIN}}$ (we don't need this step for NNC)
  - Evaluate the performance of the model on $\mathcal{D}^{\text{VAL}}$
- Choose the model with the best performance on $\mathcal{D}^{\text{VAL}}$
- Evaluate the model on $\mathcal{D}^{\text{TEST}}$

Cross-validation

What if we do not have validation data?

- We split the training data into S equal parts.
- We use each part *in turn* as a validation dataset and use the others as a training dataset.
- We choose the hyperparameter such that the model performs the best (based on average, variance, etc.)
- We re-train the model on the full training dataset with the best hyperparameter.

Figure 3: S = 5: 5-fold cross validation



Special case: when S = N, this will be leave-one-out.

But how do we choose the distances?

Distances depend on units of the features!

Preprocess data

Normalize data to have zero mean and unit standard deviation in each dimension

- Compute the means and standard deviations in each feature

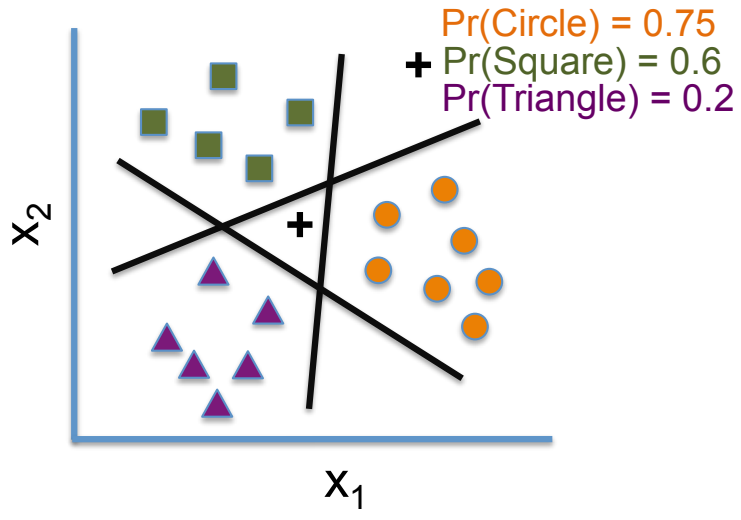
$$\bar{x}_d = \frac{1}{\mathrm{N}} \sum_n x_{nd}, \qquad s_d^2 = \frac{1}{N-1} \sum_n (x_{nd} - \bar{x}_d)^2$$

- Scale the feature accordingly

$$x_{nd} \leftarrow \frac{x_{nd} - \bar{x}_d}{s_d}$$

Many other ways of normalizing data — you would need/want to try different ones and pick among them using (cross) validation

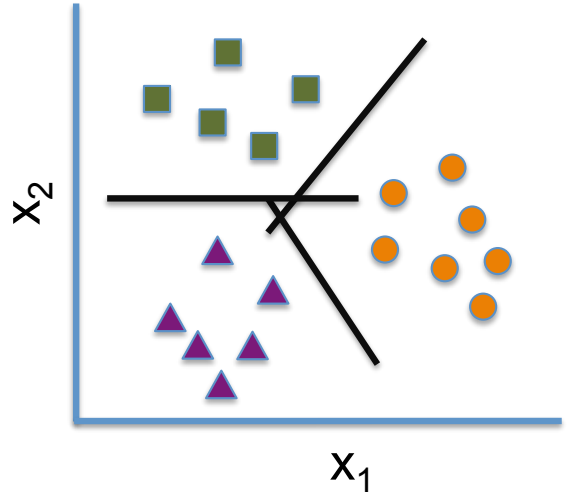
Recall: Multi-class classification



We combined binary decision boundaries to partition the feature space

- One-versus-all approach

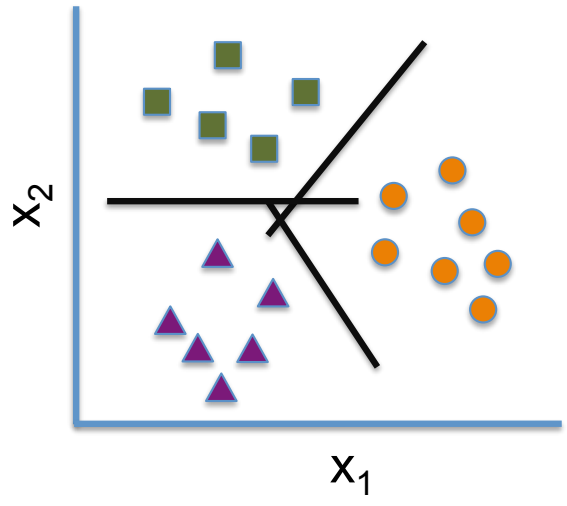
Recall: Multi-class classification



We combined binary decision boundaries to partition the feature space

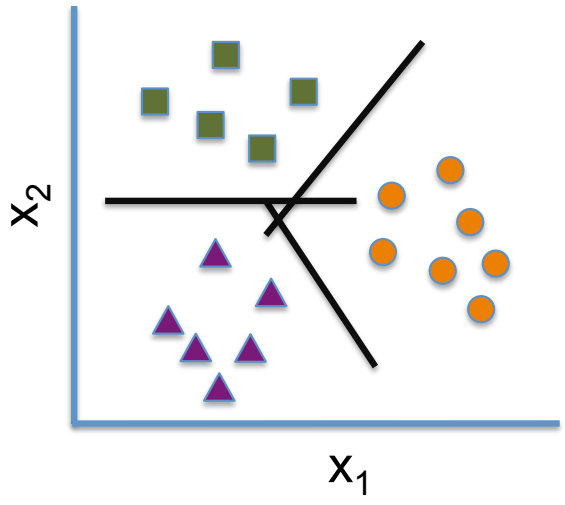
- One-versus-one approach

Recall: Multi-class classification



- Suppose the 3 classes are 3 possible treatments for an illness and you recommend treatment 1.
- The patient sues you and your lawyer needs to explain the reasoning behind the decision in court. What would she say?
  - "$\mathbf{w}_{(1)}^\top \mathbf{x} > 0$ and $\mathbf{w}_{(2)}^\top \mathbf{x} < 0$"? This might not convince the judge.
  - "Treatment 1 worked for similar patients"? This ignores the structure of your data.

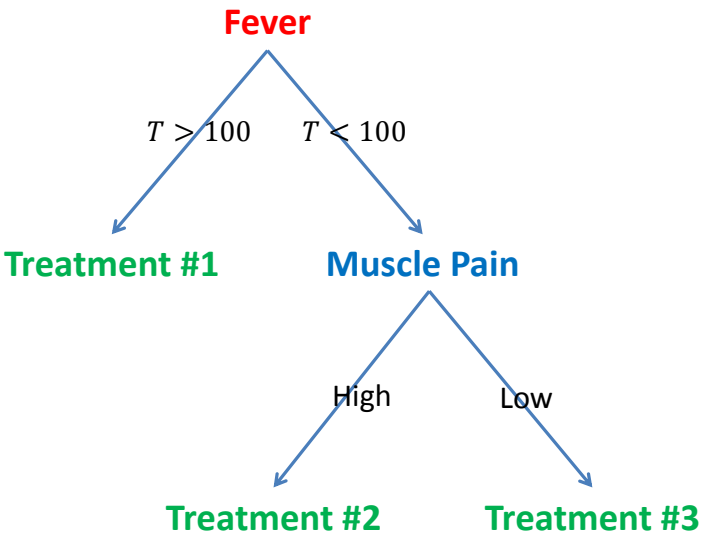
Need interpretable decision boundaries



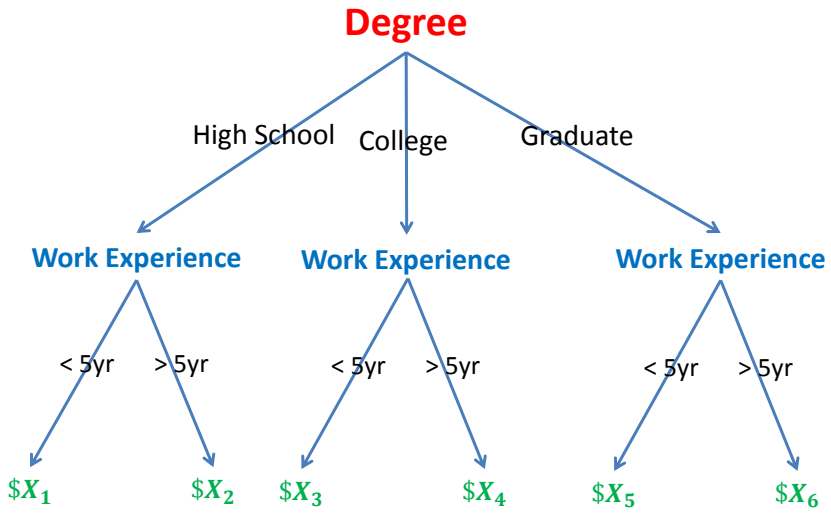
- Should be able to explain the reasoning in clear terms, e.g., "I always recommend treatment 1 when a patient has fever $\geq 100F$"
- The rules that you use to make decisions can be easily used by a lay-person without performing complex computations
- Decision trees can provide such simple decision rules

Many decisions are tree structured

Medical treatment

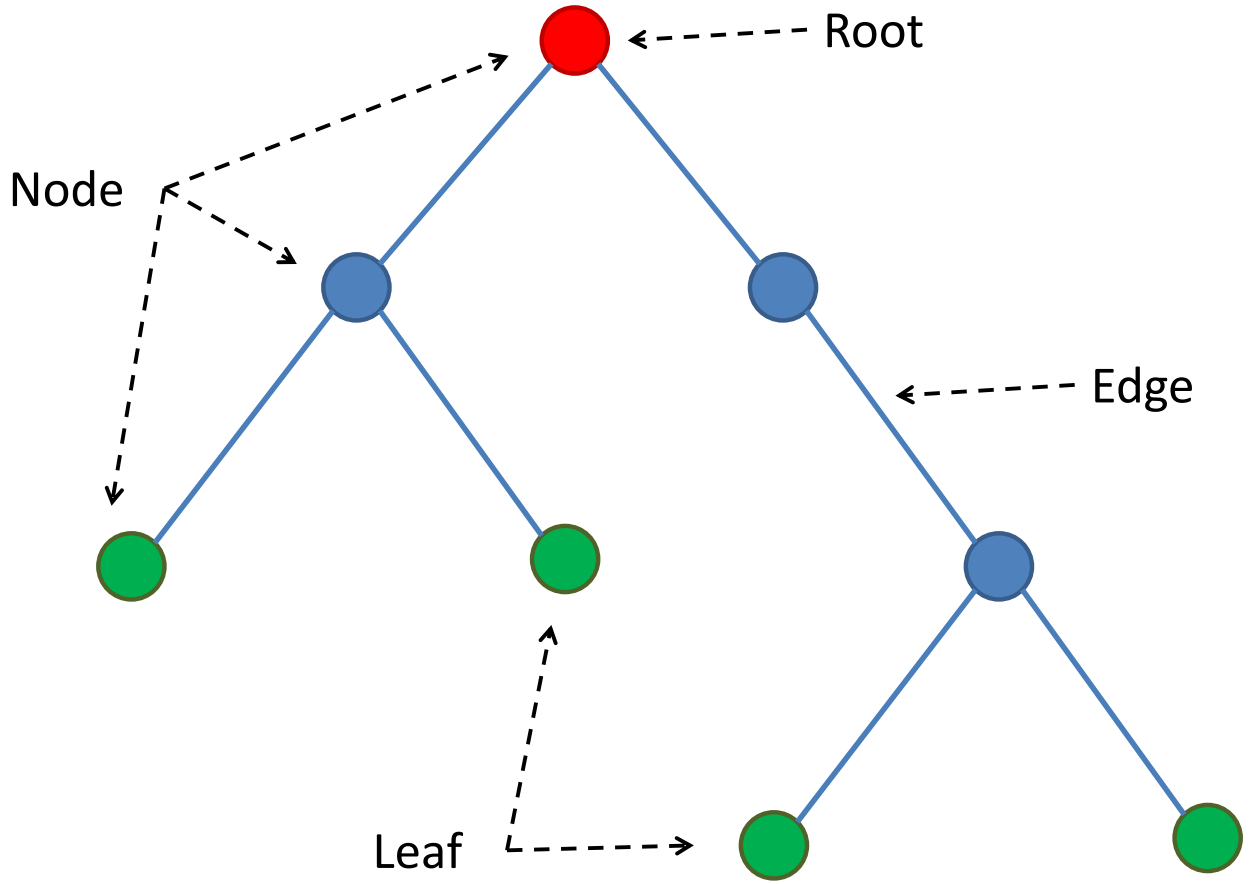


Salary in a company

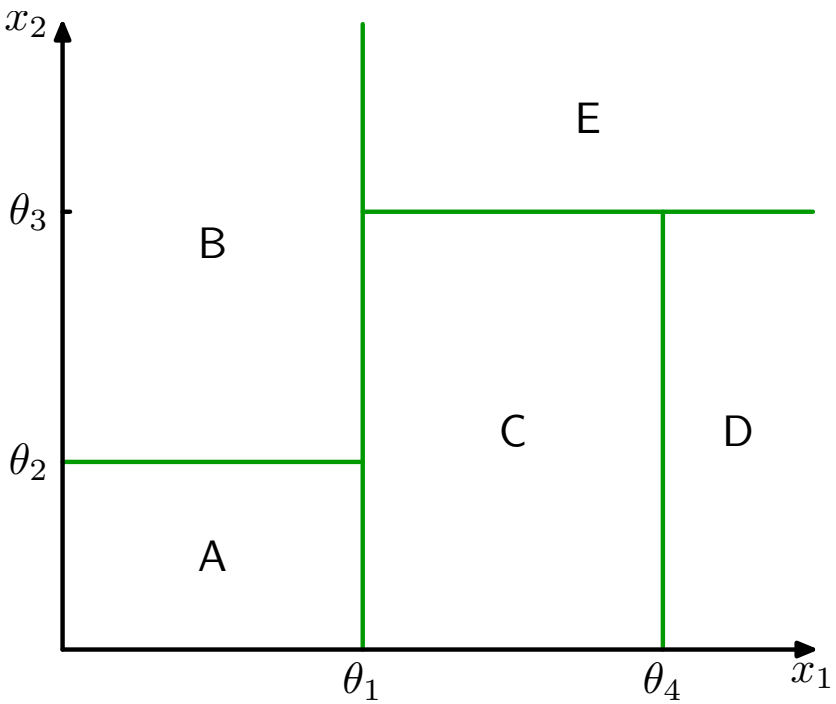


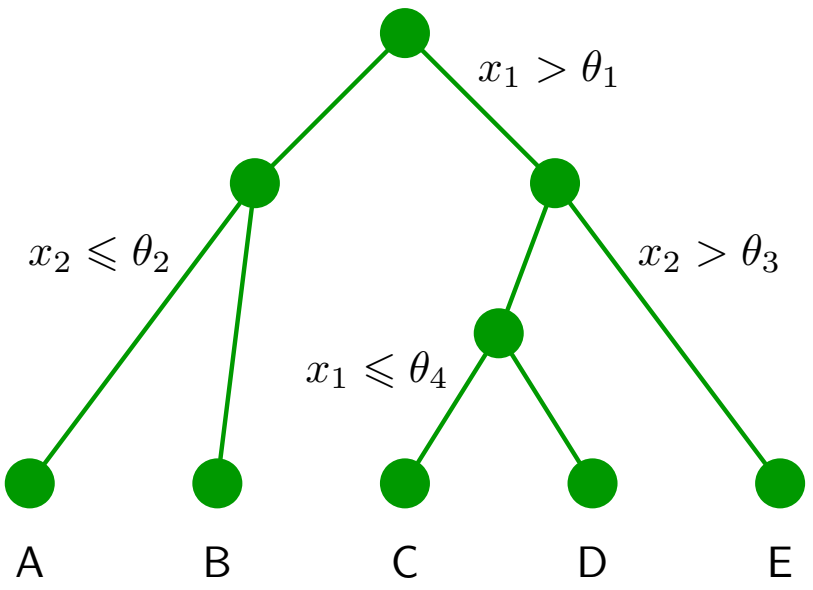
Other examples: fault detection in manufacturing systems, student admissions decisions, jail/parole decisions

Special names for nodes in a tree

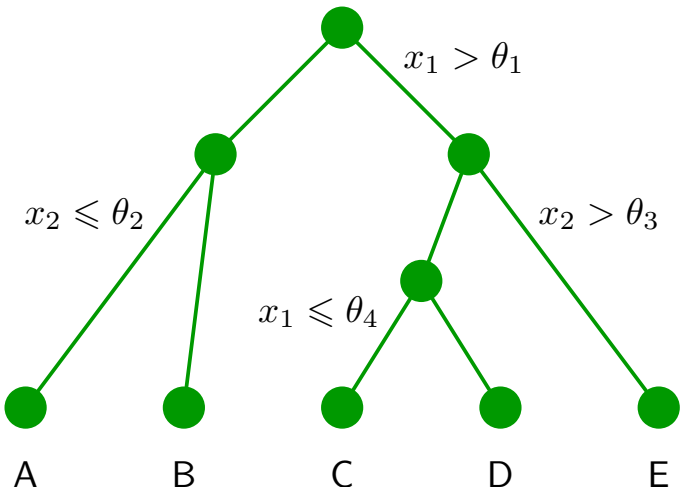


A tree partitions the feature space





Learning a tree model



Three things to learn:

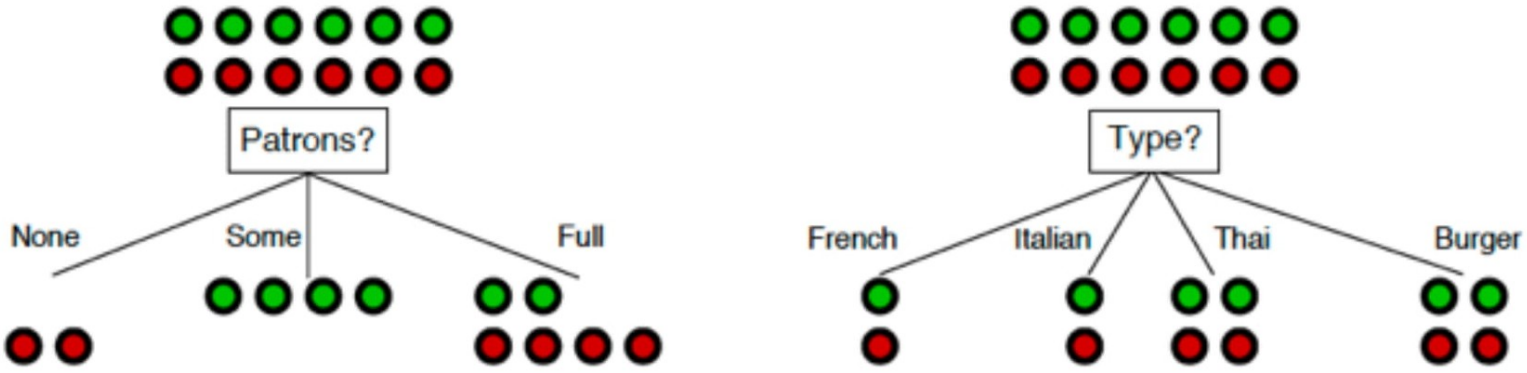
1. The structure of the tree.
2. The threshold values (θ_i).
3. The values for the leaves ($A, B, \ldots$).

Example: Choosing whether you want to wait at a restaurant

Attributes										Target
Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
T	*F*	*F*	*T*	*Some*	*$$$*	*F*	*T*	*French*	*0–10*	*T*
T	*F*	*F*	*T*	*Full*	*$*	*F*	*F*	*Thai*	*30–60*	*F*
F	*T*	*F*	*F*	*Some*	*$*	*F*	*F*	*Burger*	*0–10*	*T*
T	*F*	*T*	*T*	*Full*	*$*	*F*	*F*	*Thai*	*10–30*	*T*
T	*F*	*T*	*F*	*Full*	*$$$*	*F*	*T*	*French*	*>60*	*F*
F	*T*	*F*	*T*	*Some*	*$$*	*T*	*T*	*Italian*	*0–10*	*T*
F	*T*	*F*	*F*	*None*	*$*	*T*	*F*	*Burger*	*0–10*	*F*
F	*F*	*F*	*T*	*Some*	*$$*	*T*	*T*	*Thai*	*0–10*	*T*
F	*T*	*T*	*F*	*Full*	*$*	*T*	*F*	*Burger*	*>60*	*F*
T	*T*	*T*	*T*	*Full*	*$$$*	*F*	*T*	*Italian*	*10–30*	*F*
F	*F*	*F*	*F*	*None*	*$*	*F*	*F*	*Thai*	*0–10*	*F*
T	*T*	*T*	*T*	*Full*	*$*	*F*	*F*	*Burger*	*30–60*	*T*

Use the attributes to decide whether to wait (T) or not wait (F)

Which attribute to split first?



- Patron is a better choice – gives more information to help distinguish between the labels
- Intuition: Like playing 20 questions and choosing carefully which question to ask first
- More formally: use information gain to choose which attribute to split

How to measure information gain $I(X;Y)$?

Idea: Gaining information is equivalent to reducing our uncertainty.

- Use entropy $H(Y)$ to measure uncertainty in Y.
- We define $H(Y)$ and $H(Y|X)$ next

Definition (Entropy)

If a random variable Y takes K different values, a_1, $a_2 ... a_K$, then its entropy is

$$H[Y] = -\sum_{i=1}^{K} \Pr(Y = a_i) \log \Pr(Y = a_i)$$

Convention: $0 \log 0$ is considered as 0

Example: Entropy of a Bernoulli random variable

What is the entropy $H(Y)$ of Y, which is 1 with probability p and 0 otherwise?

Find the entropy $H(Y)$ for $p = 0.5$, $p = 0.25$, $p = 0$.

- For $p = 0.5$

  $$H(Y) = -(0.5 \log 0.5 + 0.5 \log 0.5) = \log 2 = 1 \text{ bit (log is base 2)}$$

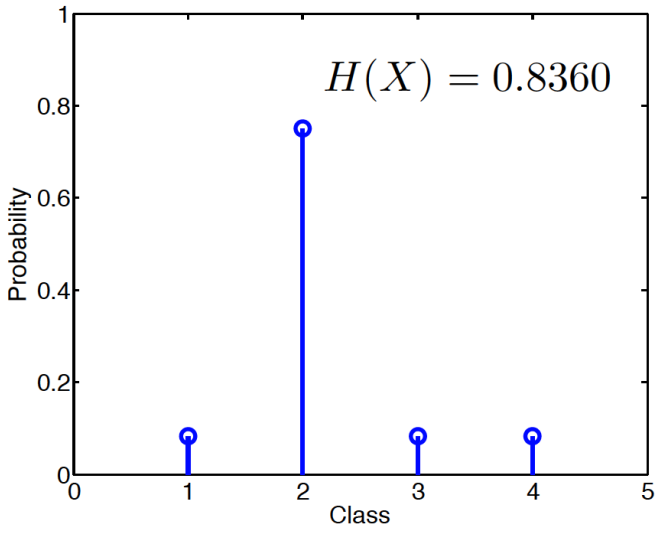
- For $p = 0.25$

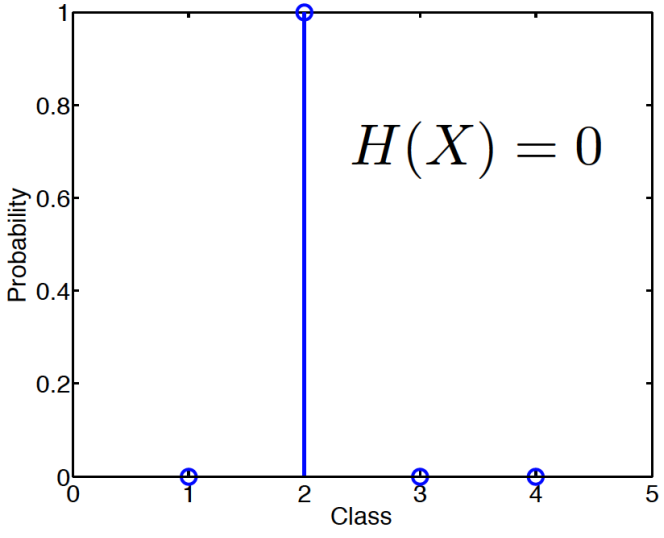
  $$H(Y) = -(0.25 \log 0.25 + 0.75 \log 0.75) = 2 \log 2 - 0.75 \log 3 = 0.81 \text{ bits}$$

- For $p = 0$, $H(Y) = 0$

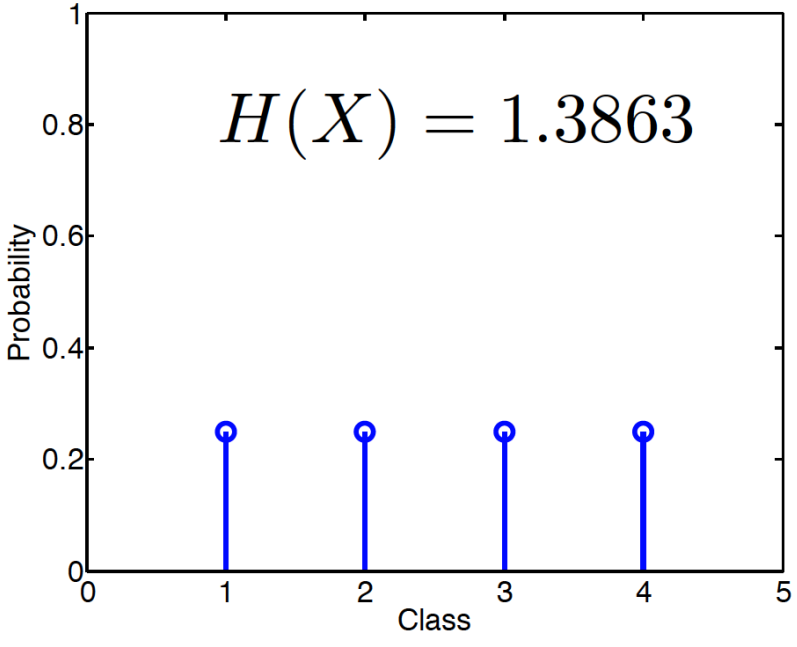
With more uncertainty ($p = 0.5$), we have a larger entropy.

Illustrating Entropy





Given a range of possible values, entropy is maximized with a uniform distribution.



Conditional entropy

Definition (Conditional Entropy)

Given two random variables X and Y

$$H[Y|X] = \sum_k P(X = a_k) H[Y|X = a_k] \tag{1}$$

In our restaurant example:

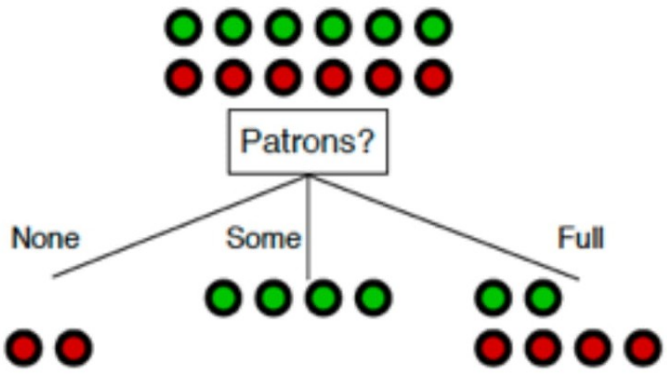
- X: the attribute to be split
- Y: wait or not (the labels)

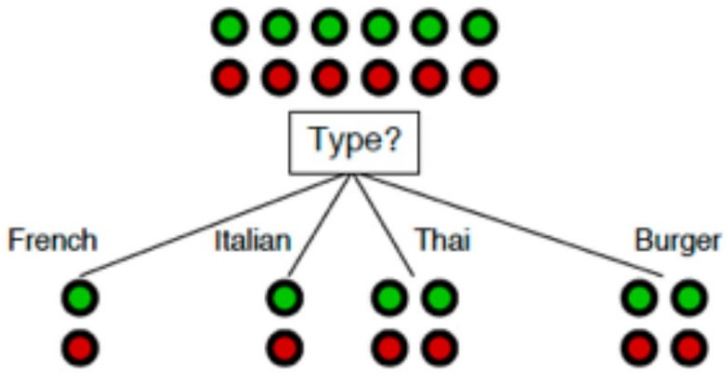
Definition (Information Gain)

$$I(X; Y) = H[Y] - H[Y|X] \tag{2}$$

Measures the reduction in entropy (i.e., the reduction of uncertainty in Y) when we also consider X.

Which attribute to split?

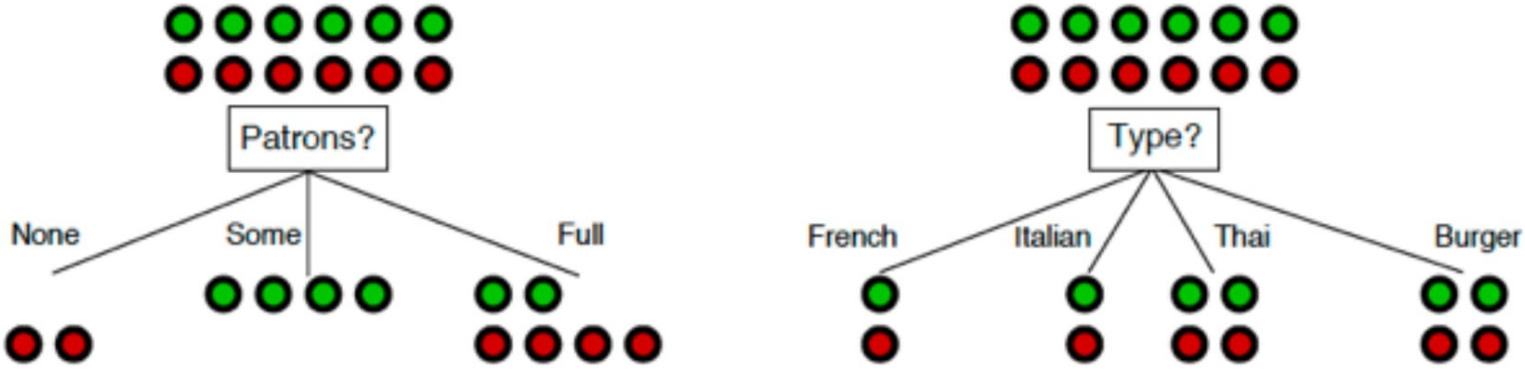




Patron vs. Type?

- Let us compute the information gain $I(X; Y) = H[Y] - H[Y|X]$ for Patron and Type
- When $H[Y]$ is fixed, we need only to compare conditional entropies

Information Gain if we split "Patron"

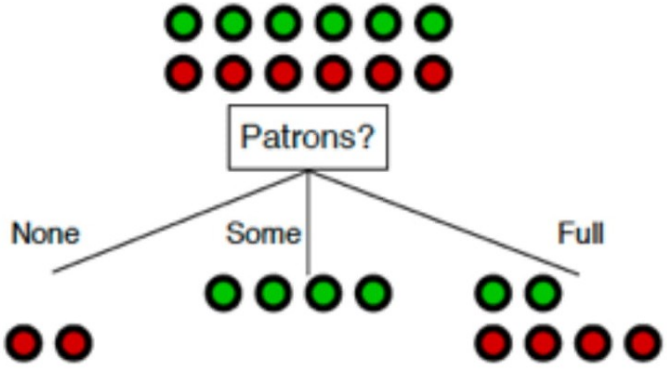


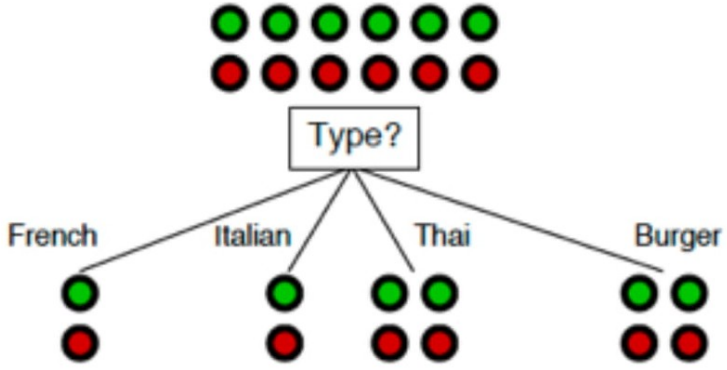
- $H(Y) = -\frac{6}{12}\log\frac{6}{12} - \frac{6}{12}\log\frac{6}{12} = 1$ bit
- $H(Y|X = none) = 0$
- $H(Y|X = some) = 0$
- $H(Y|X = full) = -\left(\frac{2}{2+4}\log\frac{2}{2+4} + \frac{4}{2+4}\log\frac{4}{2+4}\right) \approx 0.9$ bits
- Thus the conditional entropy is

$$H(Y|X) = (\frac{2}{12} \times 0 + \frac{4}{12} \times 0 + \frac{6}{12} \times 0.9) = 0.45 \text{ bits}$$

- Information Gain $I(X; Y) = 1 - 0.45 = 0.55$ bits

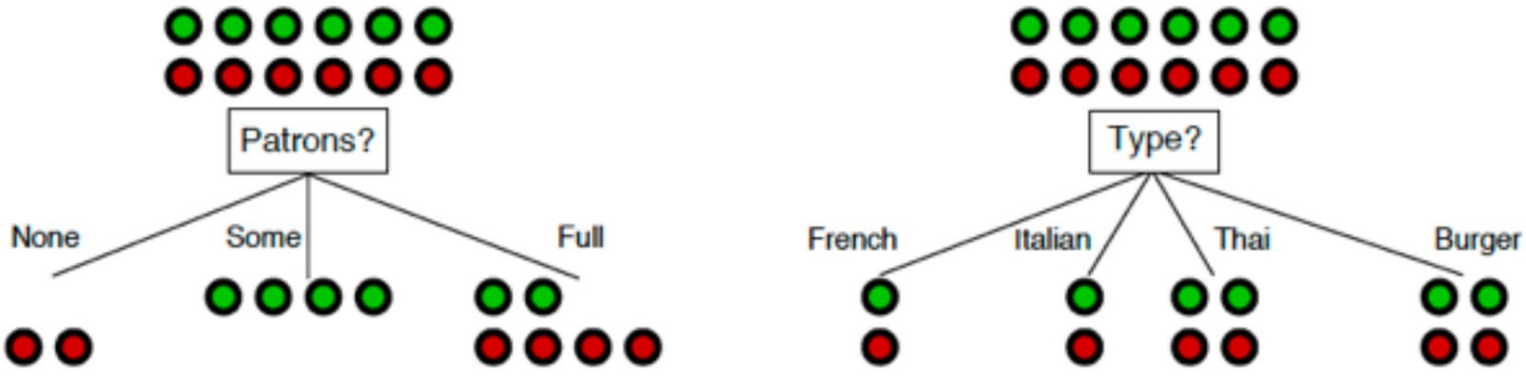
Information Gain if we split "Type"





- $H(Y) = -\frac{6}{12}\log\frac{6}{12} - \frac{6}{12}\log\frac{6}{12} = 1$ bit
- $H(Y|X = \textit{french}) = \log 2 = 1$ bit
- $H(Y|X = \textit{italian}) = \log 2 = 1$ bit
- $H(Y|X = \textit{thai}) = \log 2 = 1$ bit
- $H(Y|X = \textit{burger}) = \log 2 = 1$ bit
- Thus the conditional entropy is
  $H(Y|X) = \frac{2}{12} \times 1 + \frac{2}{12} \times 1 + \frac{4}{12} \times 1 + \frac{4}{12} \times 1 = 1$ bit
- Information Gain $I(X; Y) = 1 - 1 = 0$ bits

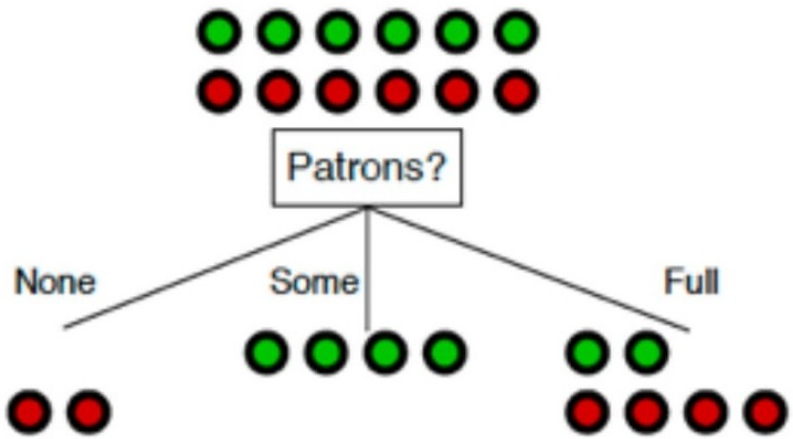
Splitting on "Patron" or "Type"?



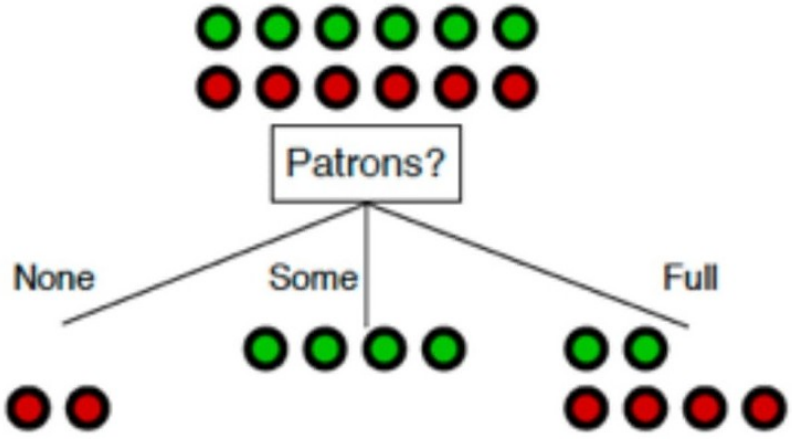
- Information gain from "Patron" is 0.55 bits.
- Information gain from "Type" is 0 bits.

Thus, we should split on "Patron" and not "Type" (higher information gain). This is consistent with our intuition.

Next split?



Do we split on "None" or "Some"?



- No, we do not
- The decision is deterministic, as seen from the training data.

Next split

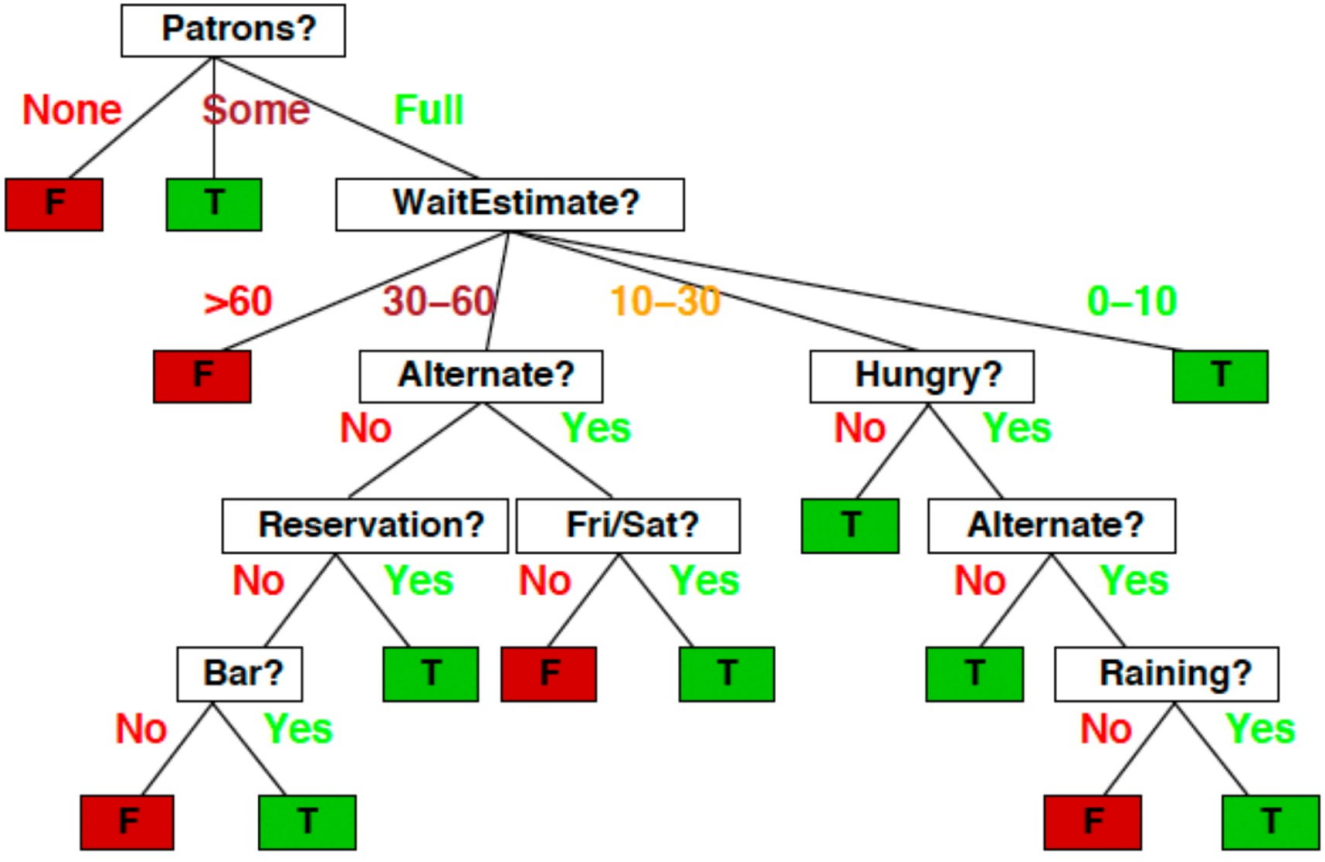
Next split?

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

Classification of examples is positive (T) or negative (F)

We will look only at the 6 instances with Patrons == Full

Greedily we build the tree and get this



What is the optimal Tree Depth?

- What happens if we pick the wrong depth?
  - If the tree is too deep, we can overfit
  - If the tree is too shallow, we underfit
- Max depth is a hyperparameter that should be tuned by the data
- Alternative strategy is to create a very deep tree, and then to prune it (see Section 9.2.2 in ESL for details)

Cost Complexity Pruning

Pruning means collapsing non-terminal nodes to eliminate a split.

Cost complexity criterion

$$C_\alpha(T) = \sum_{m=1}^{|T|} \text{error}_m(T) + \alpha|T|$$

- Find the tree T that minimizes the cost $C_\alpha(T)$, where $m = 1, 2, \ldots, |T|$ indexes the leaf nodes.
- Measure error of the training data at each leaf node as before (misclassification rate, squared error for linear regression).
- Choose α as a hyperparameter (similar to regularization).

To find the tree that minimizes C_α, greedily collapse the node in the full tree that increases the error rate the least.

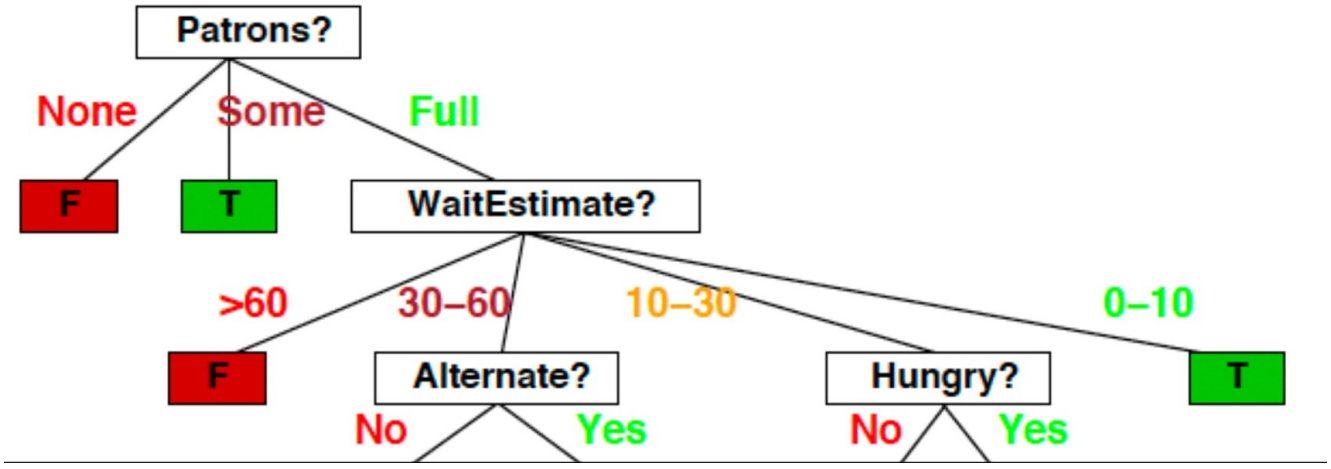
Overfitting in Decision Trees

- Including irrelevant attributes can result in overfitting the training example data.
- If we have too little training data, even a reasonable hypothesis space will overfit.

Strategies to avoid overfitting

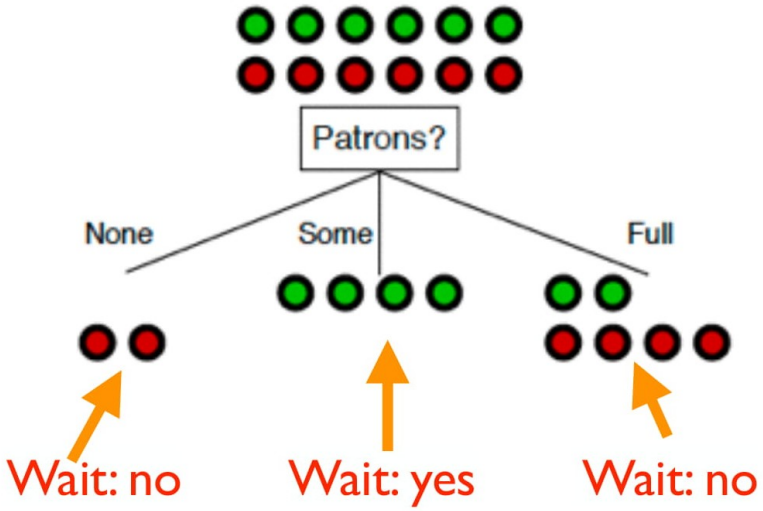
- Stop growing when data split is not statistically significant.
- Acquire more training data.
- Remove irrelevant attributes (manual process — not always possible).
- Grow full tree, then post-prune (e.g., cost complexity)

How to classify with a pruned decision tree?



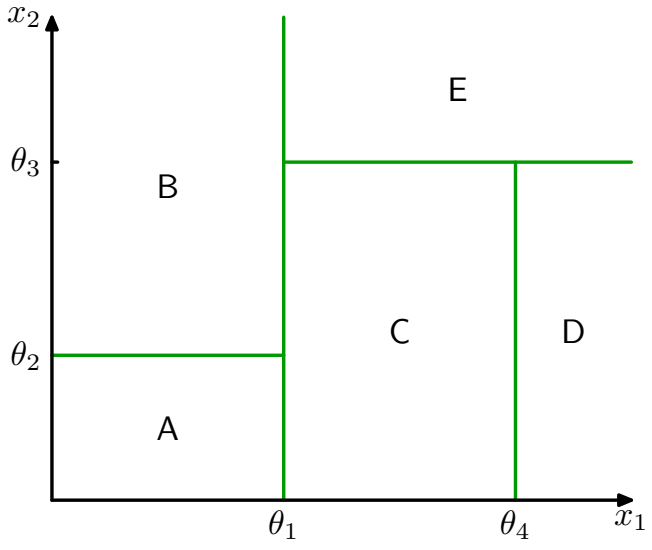
- If we stop here, not all training samples would be classified correctly
- More importantly, how do we classify a new instance?
- We label the leaves of this smaller tree with the majority of training sample's labels.

Example



Computational Considerations: Numerical Features

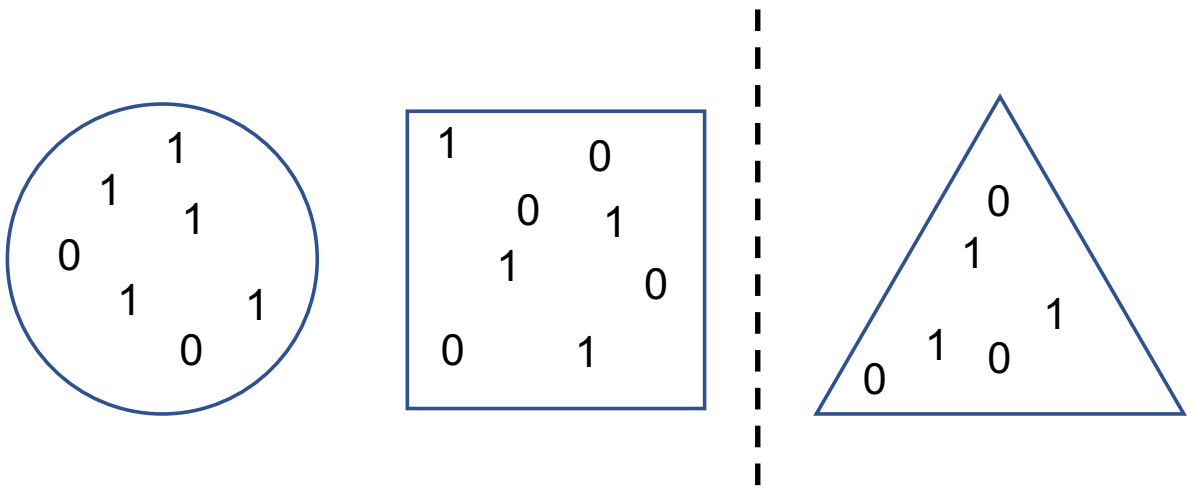
- How should we decide the threshold to use in splitting the feature?
- Can we do this efficiently?
  - Yes – for a given feature we only need to consider the n values in the training data!
  - If we sort each feature by these n values, we can quickly compute and maximize the information gain along each possible threshold.
  - This takes $O(dn \log n)$ time, where d is the number of features



$$x_1^{(1)} \quad x_1^{(2)} \quad x_1^{(3)} \;\Big|^{\theta_5}\; x_1^{(4)} \quad \cdots \quad x_1^{(n)}$$

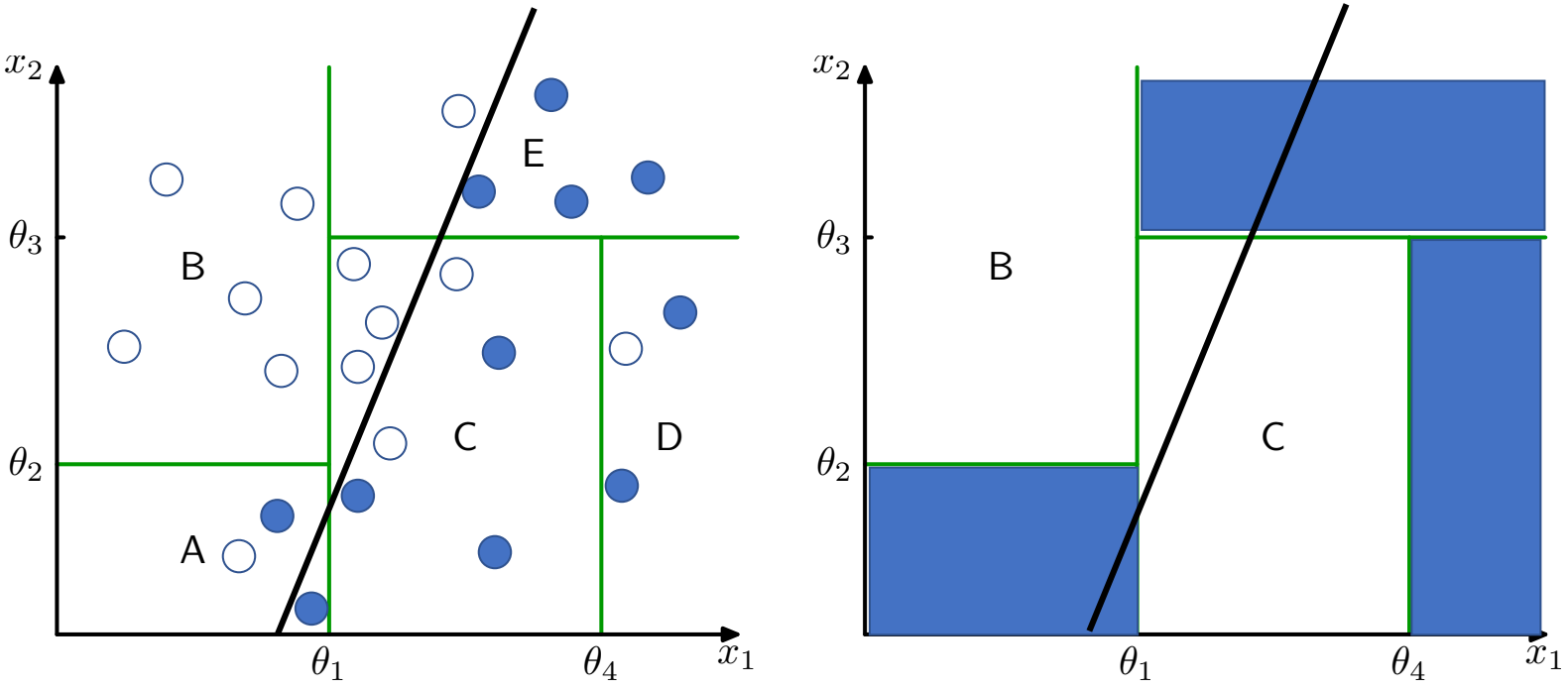
Computational Considerations: Categorical Features

- Assuming q distinct categories, there are $\frac{1}{2}\left(2^q - 2\right) = 2^{q-1} - 1$ possible partitions
- Things simplify in the case of binary classification or regression
  - Can sort the features by the fraction of labels falling in class 1
  - Suffices to consider only $q - 1$ possible splits (see Section 9.2.4 in ESL)
- Example: suppose we have two labels (0 or 1) and the feature is "shape," which has three categories (circle, square, or triangle).



Disadvantages of Decision Trees

- Binary decision trees find it hard to learn linear boundaries.
- Decision trees can have high variance due to dependence on the training data.
- We use heuristic training techniques: finding the optimal partition is NP-hard.



Advantages of Decision Trees

- Can be interpreted by humans (as long as the tree is not too big)
- Computationally efficient (for shallow trees)
- Handles both numerical and categorical data
- Can be used for both classification and regression
- Compact representation: unlike Nearest Neighbors we don't need training data at test time
- But, like NN, decision trees are nonparametric because the number of parameters depends on the data